Project Brief: Employee Leave Management System

Project Duration: 28 July 2026 to Aug 10 2026

Objective

Build a simple web-based Employee Leave Management System that demonstrates user authentication, role-based access, CRUD operations, file uploads, and a complete approval workflow. The focus is on building a functional application with clean code and a good user experience.

Functional Requirements

1. Authentication

Employee

- Users should be able to register using:
  - Username
  - Password
- No email verification or real email integration is required.
- After registration, users should be able to log in using their credentials.

Manager

A single manager account should already exist in the system.

Example credentials:

- **Username:** manager@gcu.in
- **Password:** *(Choose any password and document it for the demo.)*

No functionality should exist to create additional manager/admin accounts.

Only this predefined account should be able to access the Manager Portal.

Employee Portal

After logging in, an employee should be able to:

Dashboard

A simple dashboard is sufficient.

It should provide:

- Welcome message
- Apply Leave button

- View Leave History button

Apply Leave

The employee should be able to submit a leave request containing:

- Leave Reason
- Start Date
- End Date
- Supporting Document Upload

Supporting documents should be actual files uploaded to the backend (not just text fields).

Leave History

Employees should be able to view all leave requests they have submitted.

Each request should display:

- Leave Dates
- Reason
- Current Status
  - Pending
  - Approved
  - Rejected
- Manager's Remarks (if any)

Notifications

When a leave request is approved or rejected, the employee should receive an in-app toast notification after logging in or while using the application.

No email or SMS notifications are required.

Manager Portal

Only the predefined manager account should have access to this section.

The Manager Portal should contain the following pages.

Employees

Display a list of all registered employees.

Basic information is sufficient, such as:

- Username
- Date Joined (optional)

Leave Requests

Display all leave requests submitted by employees.

The manager should be able to:

- View leave details
- View uploaded supporting document
- Approve the request
- Reject the request
- Enter remarks/reason before approving or rejecting

The updated status should immediately be reflected for the employee.

Security Requirements

- Authentication must be implemented.
- Protected routes should prevent unauthorized users from accessing the Manager Portal.
- Employees should not be able to access manager pages.
- The predefined manager account should be the only account with manager privileges.

Technical Requirements

You may use any suitable technology stack. A recommended stack includes:

Frontend

- React

Backend

- Node.js
- Express.js

Database

- PostgreSQL
- MySQL
- Supabase
- Any equivalent relational database

Authentication may be implemented using JWT or any secure session-based approach.

Deployment

The completed application should be deployed to a free hosting platform.

Examples include:

- Vercel
- Netlify
- Render
- Railway
- Supabase

The backend and database should be fully functional.

Demo Requirements

For the final demonstration:

- The application should already contain sample employee accounts and leave requests.
- Demonstrate the complete workflow:
    1. Log in as an Employee.
    2. Submit a leave request with a supporting document.
    3. Log in as the Manager.
    4. Review the request.
    5. Approve or reject it with remarks.
    6. Log back in as the Employee and demonstrate that the status has updated and the notification is displayed.

Deliverables

- Complete Source Code (GitHub Repository)
- Live Hosted Application
- Working Backend
- Connected Database
- README with setup instructions
- Sample Manager Credentials
- Brief explanation of the project architecture and design decisions

Evaluation Criteria

Your submission will be evaluated based on:

- Functionality and completeness
- Problem-solving approach
- Code quality and project structure
- UI/UX and usability
- Secure authentication and authorization
- Database design
- File upload implementation
- Deployment quality
- Ability to explain your implementation during the final review

Note: AI tools such as ChatGPT, GitHub Copilot, Claude, Cursor, etc., may be used during development. However, every participant must be able to explain their implementation and reasoning during the final evaluation. You may be asked to modify or extend parts of your project live during the review.